

大津法

```
//-----  
-----  
// @brief      快速大津      CYH-NYY  
// @return     uint8  
// @since      v1.1  
// Sample usage: OTSU_Threshold = otsuThreshold(image_use_dvp[0]);//大津法阈  
值  
//-----  
-----  
uint8 otsuThreshold(uint8 *image)  
{  
    uint16 width=MT9V03X_W;  
    uint16 height=MT9V03X_H-4;  
    #define GrayScale 256  
    int pixelCount[GrayScale] = {0};//每个灰度值所占像素个数  
    float pixelPro[GrayScale] = {0};//每个灰度值所占总像素比例  
    int i,j;  
    int Sumpix = width * height/4;    //总像素点  
    uint8 threshold = 0;  
  
    uint8* data = image; //指向像素数据的指针  
  
    //统计灰度级中每个像素在整幅图像中的个数  
    for (i = 0; i < height; i+=2)  
    {  
        for (j = 0; j < width; j+=2)  
        {  
            pixelCount[(int)data[i * width + j]]++; //将像素值作为计数数组的下标  
            // pixelCount[(int)image[i][j]]++; 若不用指针用这个  
        }  
    }  
    float u = 0;  
    for (i = 0; i < GrayScale; i++)  
    {  
        pixelPro[i] = (float)pixelCount[i] / Sumpix;    //计算每个像素在整幅图像中的  
比例  
        u += i * pixelPro[i]; //总平均灰度  
    }  
}
```

```

float maxVariance=0.0; //最大类间方差
float w0 = 0, avgValue = 0; //w0 前景比例 , avgValue 前景平均灰度
for(int i = 0; i < 256; i++) //每一次循环都是一次完整类间方差计算 (两个 for
叠加为 1 个)
{
    w0 += pixelPro[i]; //假设当前灰度 i 为阈值, 0~i 灰度像素所占整幅图像的比例即前景比例
    avgValue += i * pixelPro[i];

    float variance = pow((avgValue/w0 - u), 2) * w0 / (1 - w0); //类间方差
    if(variance > maxVariance)
    {
        maxVariance = variance;
        threshold = (uint8)i;
    }
}

if(threshold>40 && threshold<200)
    last_threshold = threshold;
else
    threshold = last_threshold;

return threshold;
}

```

最长白列（21 届走马观碑视觉培训方案）

```

//-----
//-----
// 函数简介      寻找最长白列      CYH-NYY
// 参数说明
// 返回参数
// 使用示例      get_highest();
// 备注信息
//-----
//-----

int lwline=94;
int lw=119;
int xxx=0;

void get_highest(void)
{

```

```

lwline=94;
lw=119;
xxx=0;

for (int i=64;i<124;i+=3)          //从 64 列到 124 列从下往上找最长白列，隔
三列扫一次减少代码运行时间
{
    if(image_use[80][i]>=daj)      //每一列的 80 行为黑就不扫
    {
        for (int j=85;j>=0;j--)
        {
            //找到白黑黑跳变，消除噪点影响
            if ((image_use[j][i]>=daj&&image_use[j-1][i]<daj&&image_use[j-
2][i]<daj)||j<=5)
            {
                //贪心算法求得最长白列
                if (j<lw)
                {
                    lw=j;
                    lwline=i;
                }
                break;
            }
        }
    }
}
}

先进八邻域
//方向向量数组
int guize[8][2]={
    {0,-1},
    {1,-1},
    {1,0},
    {1,1},
    {0,1},
    {-1,1},
    {-1,0},
    {-1,-1}
};

//-----
-----

// 函数简介      黑白判断函数，八邻域爬线所需      CYH-NYY
// 参数说明
// 返回参数      x,y 点 k 方向是黑还是白

```

```

// 使用示例      cbh(x 坐标,y 坐标,k 方向);
// 备注信息
//-----
-----

//
float L_edge[100][2];      //左边界结构体
float Lout_edge[100][2];   //左边界结构体
float Luse_edge[100][2];
float R_edge[100][2];      //右边界结构体
float Rout_edge[100][2];   //左边界结构体
float Ruse_edge[100][2];
int L_count=0, R_count = 0,Lout_count=0,Rout_count=0;           //左右
边点的个数
int L_amount = 100, R_amount = 100; //左右边界搜点时最多允许的点

int hig=18;              //起始扫线行上移行数

uint8 cbh(int x,int y,int k)
{
    int i=0,j=0;
    i=x+guize[k][0];
    j=y+guize[k][1];

    if (j>=MT9V03X_H-hig) return 1;           //底部黑
框
    else if (j<=1||i<=3||i>=MT9V03X_W-4)return 2;           //扫弦停止框
    else if (image_use[j][i]>=daj) return 0;           //白点
    else return 1;
//黑点
}
//-----
-----

// 函数简介      八邻域爬线      CYH-NYY
// 参数说明
// 返回参数
// 使用示例      get_BLY(输出左边界数组, 输出左边界点数, 输出右边界数组, 输出
右边界点数, 起始扫线行上移行数);
// 备注信息
//-----
-----

//

void get_BLY(float L_line[][2],int *Lnum,float R_line[][2],int *Rnum,int len)

```

```

{
    hig=len;
    int L_C=0,R_C=0;

    //爬初始点-----

    for (int16 i=(int16)lwline;i>0;i--)
    {
        if (cbh(i,MT9V03X_H-hig-1,6)==1&&cbh(i+1,MT9V03X_H-hig-
1,6)==1&&cbh(i+2,MT9V03X_H-hig-1,6)==0)
        {
            L_line[0][0]=i+2;
            break;
        }
        else if (cbh(i+1,MT9V03X_H-hig-1,6)==2)
        {
            L_line[0][0]=i+2;
            break;
        }
    }

    for (int16 i=(int16)lwline;i<MT9V03X_W;i++)
    {
        if (cbh(i,MT9V03X_H-hig-1,2)==1&&cbh(i-1,MT9V03X_H-hig-
1,2)==1&&cbh(i-2,MT9V03X_H-hig-1,2)==0)
        {
            R_line[0][0]=i-2;
            break;
        }
        else if (cbh(i-1,MT9V03X_H-hig-1,2)==2)
        {
            R_line[0][0]=i-2;
            break;
        }
    }

    L_line[0][1]=(float)MT9V03X_H-hig-1;
    R_line[0][1]=(float)MT9V03X_H-hig-1;

    L_C=1;
    R_C=1;

    int l_dirl=6,l_dirr=2;

```

```

int dirl=6,dirr=2;
//左边界-----

for (int i=1;i<L_amount;i++)
{
    int cishu=0;

    if (cbh(L_line[i-1][0],L_line[i-1][1],0)==2||cbh(L_line[i-1][0],L_line[i-1][1],1)==2||
        cbh(L_line[i-1][0],L_line[i-1][1],2)==2||cbh(L_line[i-1][0],L_line[i-1][1],3)==2||
        cbh(L_line[i-1][0],L_line[i-1][1],4)==2||cbh(L_line[i-1][0],L_line[i-1][1],5)==2||
        cbh(L_line[i-1][0],L_line[i-1][1],6)==2||cbh(L_line[i-1][0],L_line[i-1][1],7)==2) break;

    for (int j=0;j<8;j++)
    {
        if (cbh(L_line[i-1][0],L_line[i-1][1],(j+l_dirl+4)%8)==1&&
            cbh(L_line[i-1][0],L_line[i-1][1],(j+l_dirl+4+1)%8)==0&&
            cbh(L_line[i-1][0],L_line[i-1][1],(j+l_dirl+4+2)%8)==0)
        {
            dirl=(j+dirl+4+1)%8;
            break;
        }
        else
        {
            cishu++;
        }
    }
    l_dirl=dirl;

    if (cishu==8)
    {
        break;
    }
    else
    {
        L_line[i][0]=L_line[i-1][0]+guize[dirl][0];
        L_line[i][1]=L_line[i-1][1]+guize[dirl][1];
    }
    L_C++;
}

//右边界-----

```

```

for (int i=1;i<R_amount;i++)
{
    int cishu=0;

    if (cbh(R_line[i-1][0],R_line[i-1][1],0)==2||cbh(R_line[i-1][0],R_line[i-
1][1],1)==2||
        cbh(R_line[i-1][0],R_line[i-1][1],2)==2||cbh(R_line[i-1][0],R_line[i-
1][1],3)==2||
        cbh(R_line[i-1][0],R_line[i-1][1],4)==2||cbh(R_line[i-1][0],R_line[i-
1][1],5)==2||
        cbh(R_line[i-1][0],R_line[i-1][1],6)==2||cbh(R_line[i-1][0],R_line[i-
1][1],7)==2) break;

    for (int j=0;j<8;j++)
    {
        if (cbh(R_line[i-1][0],R_line[i-1][1],(8-j+l_dirr+4)%8)==1&&
            cbh(R_line[i-1][0],R_line[i-1][1],(8-j+l_dirr+4-1)%8)==0&&
            cbh(R_line[i-1][0],R_line[i-1][1],(8-j+l_dirr+4-2)%8)==0)
        {
            dirr=(8-j+l_dirr+4-1)%8;
            break;
        }
        else
        {
            cishu++;
        }
    }
    l_dirr=dirr;

    if (cishu==8)
    {
        break;
    }
    else
    {
        R_line[i][0]=R_line[i-1][0]+guize[dirr][0];
        R_line[i][1]=R_line[i-1][1]+guize[dirr][1];
    }
    R_C++;
}

*Lnum=L_C;
*Rnum=R_C;

```

```

}
三角滤波
// 点集三角滤波    CYH-NYY
void blur_points(float img_in[][2], float img_out[][2], int num, int kernel)
{
    int half = kernel / 2;
    for (int i = 0; i < num; i++)
    {
        imag_out[i][0] = imag_out[i][1] = 0;
        for (int j = -half; j <= half; j++)
        {
            int fu=i+j;
            if (fu<0)fu=0;
            if (fu>num-1)fu=num-1;
            imag_out[i][0] += img_in[fu][0] * (half + 1 - abs(j));
            imag_out[i][1] += img_in[fu][1] * (half + 1 - abs(j));
        }
        imag_out[i][0] /= (2 * half + 2) * (half + 1) / 2;
        imag_out[i][1] /= (2 * half + 2) * (half + 1) / 2;
    }
}

```

重采样

```

//-----
-----
// 函数简介      等距采样      CYH-NYY
// 参数说明
// 返回参数
// 使用示例      resample_points2(输入点集,输入点集数量,输出点集,输出点集数量,点
间距);
// 备注信息
//-----
-----

```

```

void resample_points2(float pts_in[][2], int num1, float pts_out[][2], int *num2, float dist)
{
    if (num1 < 0)
    {
        *num2 = 0;
        return;
    }
    pts_out[0][0] = pts_in[0][0];
    pts_out[0][1] = pts_in[0][1];
    int len = 1;
    for (int i = 0; i < num1 - 1 && len < *num2; i++) {

```



```

float x0 = pts_in[i][0];
float y0 = pts_in[i][1];
float x1 = pts_in[i + 1][0];
float y1 = pts_in[i + 1][1];

do {
    float x = pts_out[len - 1][0];
    float y = pts_out[len - 1][1];

    float dx0 = x0 - x;
    float dy0 = y0 - y;
    float dx1 = x1 - x;
    float dy1 = y1 - y;

    float dist0 = sqrt(dx0 * dx0 + dy0 * dy0);
    float dist1 = sqrt(dx1 * dx1 + dy1 * dy1);

    float r0 = (dist1 - dist) / (dist1 - dist0);
    float r1 = 1 - r0;

    if (r0 < 0 || r1 < 0) break;
    x0 = x0 * r0 + x1 * r1;
    y0 = y0 * r0 + y1 * r1;
    pts_out[len][0] = x0;
    pts_out[len][1] = y0;
    len++;
} while (len < *num2);

}
*num2 = len;
}
求斜率/中线
//-----
-----
// 函数简介      获取中线      CYH-NYY
// 参数说明
// 返回参数
// 使用示例      get_mid(输入边线,输出边线,边线点数量);
// 备注信息
//-----
-----
float Lin_MID[50][2];      //左中线结构体
float Lout_MID[50][2];
float Rin_MID[50][2];      //右中线结构体

```

```

float Rout_MID[50][2];
int Lmidnum=0,Lmidnum2=0;
int Rmidnum=0,Rmidnum2=0;

void get_mid_L(float line[][2],int num,float mid[][2],int *num2,int approx_num)
{
    int mnum=1;
    float dist=rood_wid/2;
    mid[0][0]=MT9V03X_W/2-2;
    mid[0][1]=MT9V03X_H-2;
    for (int i = 0; i < num; i++)
    {
        float dx = line[clip(i + approx_num, 0, num - 1)][0] - line[clip(i - approx_num,
0, num - 1)][0];
        float dy = line[clip(i + approx_num, 0, num - 1)][1] - line[clip(i - approx_num,
0, num - 1)][1];
        float dn = sqrt(dx * dx + dy * dy);
        dx /= dn;
        dy /= dn;
        if (line[i][1] + dx * dist>MT9V03X_H-2) continue;
        else
        {
            mid[mnum][0] = line[i][0] - dy * dist;
            mid[mnum][1] = line[i][1] + dx * dist;
        }
        mnum++;
    }
    *num2=mnum;
}

```

```

void get_mid_R(float line[][2],int num,float mid[][2],int *num2,int approx_num)
{
    int mnum=1;
    float dist=rood_wid/2;
    mid[0][0]=MT9V03X_W/2-2;
    mid[0][1]=MT9V03X_H-2;
    for (int i = 0; i < num; i++)
    {
        float dx = line[clip(i + approx_num, 0, num - 1)][0] - line[clip(i - approx_num,
0, num - 1)][0];
        float dy = line[clip(i + approx_num, 0, num - 1)][1] - line[clip(i - approx_num,
0, num - 1)][1];
        float dn = sqrt(dx * dx + dy * dy);
        dx /= dn;

```

```

        dy /= dn;
        if (line[i][1] - dx * dist > MT9V03X_H - 2) continue;
        else
        {
            mid[mnum][0] = line[i][0] + dy * dist;
            mid[mnum][1] = line[i][1] - dx * dist;
        }
        mnum++;
    }
    *num2 = mnum;
}

```

```

int clip(int x, int low, int up) {
    return x > up ? up : x < low ? low : x;
}

```

```

float fclip(float x, float low, float up) {
    return x > up ? up : x < low ? low : x;
}

```

求夹角

```

//-----
//-----
// 函数简介      求夹角 CYH-NYY
// 参数说明
// 返回参数
// 使用示例      local_angle_points(输入点集,输入点集数量,输出角集, 点间距);
// 备注信息
//-----
//-----

```

```

void local_angle_points(float pts_in[][2], int num, float angle_out[], int dist)
{
    for (int i = 0; i < num; i++) {
        if (i <= 0 || i >= num - 1) {
            angle_out[i] = 0;
            continue;
        }
        float dx1 = pts_in[i][0] - pts_in[clip(i - dist, 0, num - 1)][0];
        float dy1 = pts_in[i][1] - pts_in[clip(i - dist, 0, num - 1)][1];
        float dn1 = sqrtf(dx1 * dx1 + dy1 * dy1);
    }
}

```

```

        float dx2 = pts_in[clip(i + dist, 0, num - 1)][0] - pts_in[i][0];
        float dy2 = pts_in[clip(i + dist, 0, num - 1)][1] - pts_in[i][1];
        float dn2 = sqrtf(dx2 * dx2 + dy2 * dy2);
        float c1 = dx1 / dn1;
        float s1 = dy1 / dn1;
        float c2 = dx2 / dn2;
        float s2 = dy2 / dn2;
        angle_out[i] = atan2f(c1 * s2 - c2 * s1, c2 * c1 + s2 * s1);
    }
}
// 角度变化率非极大抑制    CYH-NYY
void nms_angle(float angle_in[], int num, float angle_out[], int kernel)
{
    int half = kernel / 2;
    for (int i = 0; i < num; i++) {
        angle_out[i] = angle_in[i];
        for (int j = -half; j <= half; j++) {
            if (fabs(angle_in[clip(i + j, 0, num - 1)]) > fabs(angle_out[i])) {
                angle_out[i] = 0;
                break;
            }
        }
    }
}

```

9.求拐点和直道

CYH-NYY

```

void guai_mum(float Edge[][2], int num, float ang_out[], int *posi, int *if_zhi)
{
    conmax=0;
    *posi=0;
    *if_zhi=1;
    float ang_in[num];
    local_angle_points(Edge,num,ang_in,5);
    nms_angle(ang_in,num,ang_out,10);

    for (int i=0;i<num;i++)
    {
        if (ang_out[i]==0)continue;
        int im1 = clip(i - 5, 0, num - 1);
        int ip1 = clip(i + 5, 0, num - 1);
    }
}

```

```

        float conf = fabs(ang_out[i]) - (fabs(ang_out[im1]) + fabs(ang_out[ip1])) /
2;
        if (conf>1.05&&conf<2)
        {
            *posi=i;
        }
        if (*posi>0)
        {
            break;
        }
    }

/*最小二乘法判断直道*/
*if_zhi=0;
if (num>=25)
{
    float x1=Edge[2][0];
    float y1=Edge[2][1];
    float x2=Edge[num-3][0];
    float y2=Edge[num-3][1];
    float A=1/(x1-x2);
    float B=1/(y2-y1);
    float C=y2/(y1-y2)-x2/(x1-x2);
    float D=0;

    for (int i=3;i<num-3;i++)
    {
        float x0=Edge[i][0];
        float y0=Edge[i][1];
        D+=fabs(A*x0+B*y0+C)*fabs(A*x0+B*y0+C)/(A*A+B*B)/(num-6);
    }

    conmax=D;

    if (D<=2.5)
    {
        *if_zhi=1;
    }
}
}

```

十字补线和最小二乘法

```
#include "image.h"
```

```

/**
 * @brief 最小二乘法
 * @param uint8 begin          输入起点
 * @param uint8 end            输入终点
 * @param uint8 *border        输入需要计算斜率的边界首地址
 * @see CTest Slope_Calculate(start, end, border); //斜率
 * @return 返回说明
 *      -<em>>false</em> fail
 *      -<em>true</em> succeed
 */
float Slope_Calculate(uint8 begin, uint8 end, uint8 *border)
{
    float xsum = 0, ysum = 0, xysum = 0, x2sum = 0;
    int16 i = 0;
    float result = 0;
    static float resultlast;

    for (i = begin; i < end; i++)
    {
        xsum += i;
        ysum += border[i];
        xysum += i * (border[i]);
        x2sum += i * i;
    }

    if ((end - begin)*x2sum - xsum * xsum) //判断除数是否为零
    {
        result = ((end - begin)*xysum - xsum * ysum) / ((end - begin)*x2sum - xsum *
xsum);
        resultlast = result;
    }
    else
    {
        result = resultlast;
    }
    return result;
}

/**
 * @brief 计算斜率截距
 * @param uint8 start          输入起点
 * @param uint8 end            输入终点
 * @param uint8 *border        输入需要计算斜率的边界

```

```

* @param float *slope_rate      输入斜率地址
* @param float *intercept      输入截距地址
* @see CTest      calculate_s_i(start, end, r_border, &slope_rate, &intercept);
* @return 返回说明
*      -<em>>false</em> fail
*      -<em>true</em> succeed
*/
void calculate_s_i(uint8 start, uint8 end, uint8 *border, float *slope_rate, float *intercept)
{
    uint16 i, num = 0;
    uint16 xsum = 0, ysum = 0;
    float y_average, x_average;

    num = 0;
    xsum = 0;
    ysum = 0;
    y_average = 0;
    x_average = 0;
    for (i = start; i < end; i++)
    {
        xsum += i;
        ysum += border[i];
        num++;
    }

    //计算各个平均数
    if (num)
    {
        x_average = (float)(xsum / num);
        y_average = (float)(ysum / num);
    }

    /*计算斜率*/
    *slope_rate = Slope_Calculate(start, end, border); //斜率
    *intercept = y_average - (*slope_rate)*x_average; //截距
}

/**
* @brief 十字补线函数
* @param uint8(*image)[image_w]      输入二值图像
* @param uint8 *l_border      输入左边界首地址
* @param uint8 *r_border      输入右边界首地址
* @param uint16 total_num_l      输入左边循环总次数

```

```

* @param uint16 total_num_r      输入右边循环总次数
* @param uint16 *dir_l          输入左边生长方向首地址
* @param uint16 *dir_r          输入右边生长方向首地址
* @param uint16(*points_l)[2]   输入左边轮廓首地址
* @param uint16(*points_r)[2]   输入右边轮廓首地址
* @see CTest      cross_fill(image,l_border, r_border, data_statics_l, data_statics_r,
dir_l, dir_r, points_l, points_r);
* @return 返回说明
*      -<em>>false</em> fail
*      -<em>>true</em> succeed
*/
void cross_fill(uint8(*image)[image_w], uint8 *l_border, uint8 *r_border, uint16
total_num_l, uint16 total_num_r,
uint16 *dir_l, uint16 *dir_r, uint16(*points_l)[2],
uint16(*points_r)[2])
{
    uint8 i;
    uint8 break_num_l = 0;
    uint8 break_num_r = 0;
    uint8 start, end;
    float slope_l_rate = 0, intercept_l = 0;
    //出十字
    for (i = 1; i < total_num_l; i++)
    {
        if (dir_l[i - 1] == 4 && dir_l[i] == 4 && dir_l[i + 3] == 6 && dir_l[i + 5] == 6 &&
dir_l[i + 7] == 6)
        {
            break_num_l = points_l[i][1]; //传递 y 坐标
            printf("brea_knum-L:%d\n", break_num_l);
            printf("l:%d\n", i);
            printf("十字标志位: 1\n");
            break;
        }
    }
    for (i = 1; i < total_num_r; i++)
    {
        if (dir_r[i - 1] == 4 && dir_r[i] == 4 && dir_r[i + 3] == 6 && dir_r[i + 5] == 6 &&
dir_r[i + 7] == 6)
        {
            break_num_r = points_r[i][1]; //传递 y 坐标
            printf("brea_knum-R:%d\n", break_num_r);
            printf("l:%d\n", i);
            printf("十字标志位: 1\n");
            break;
        }
    }
}

```



```

    }
}
if (break_num_l && break_num_r && image[image_h - 1][4] && image[image_h -
1][image_w - 4])//两边生长方向都符合条件
{
    //计算斜率
    start = break_num_l - 15;
    start = limit_a_b(start, 0, image_h);
    end = break_num_l - 5;
    calculate_s_i(start, end, l_border, &slope_l_rate, &intercept_l);
    //printf("slope_l_rate:%d\nintercept_l:%d\n", slope_l_rate, intercept_l);
    for (i = break_num_l - 5; i < image_h - 1; i++)
    {
        l_border[i] = slope_l_rate * (i) + intercept_l; //y = kx + b
        l_border[i] = limit_a_b(l_border[i], border_min, border_max); //限幅
    }
    //计算斜率
    start = break_num_r - 15; //起点
    start = limit_a_b(start, 0, image_h); //限幅
    end = break_num_r - 5; //终点
    calculate_s_i(start, end, r_border, &slope_r_rate, &intercept_r);
    //printf("slope_r_rate:%d\nintercept_r:%d\n", slope_r_rate, intercept_r);
    for (i = break_num_r - 5; i < image_h - 1; i++)
    {
        r_border[i] = slope_r_rate * (i) + intercept_r;
        r_border[i] = limit_a_b(r_border[i], border_min, border_max);
    }
}
}
}

```